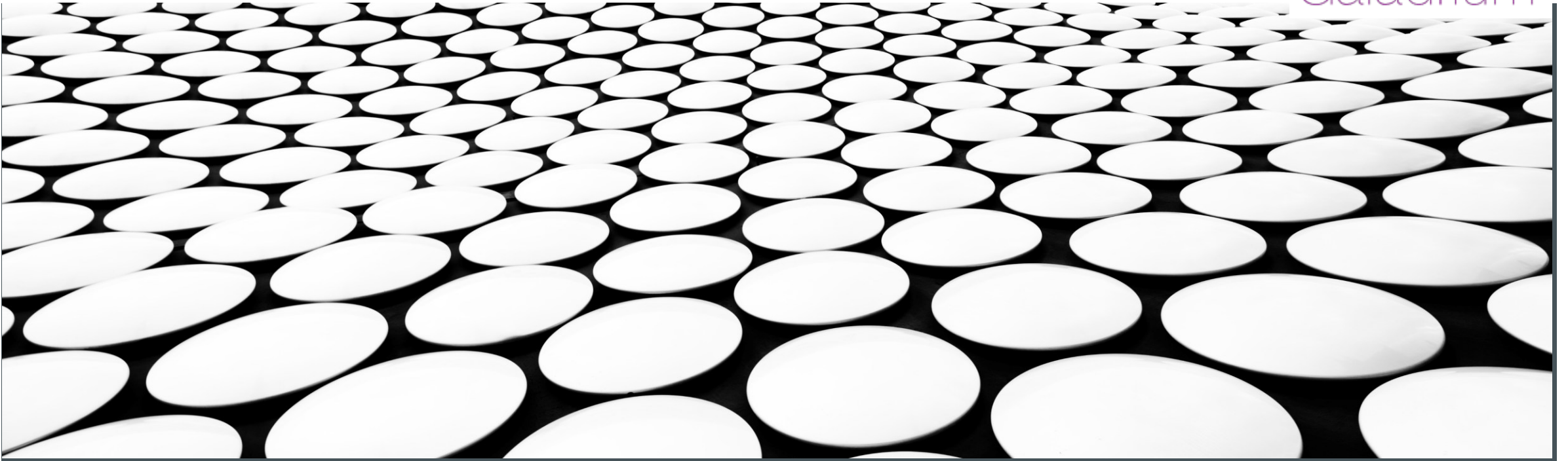


ATTECH TRAINING: IMPLEMENTATION GUIDELINES

AMHS/SWIM GATEWAY: MODULE 3

Galadrium



AMHS

Development of an ASG Using the Isode X.400 Gateway API



IMPLEMENTATION GUIDELINES

- You have already developed an AFTN/AMHS Gateway that complies with ICAO Doc 9880 Edition 2
- I presume that you are still targeting ICAO Doc 9880 Edition 2, and not edition 3
- This should be the easy part for Attech Vietnam, as lot of the code can be reused
- ICAO Doc 9880 Edition 3 adds all the complexity of AMHS Security and will need the unpublished R19.2 version

USE OF THE ISODE X.400 GATEWAY API

- The choice of what to use for the “AMHS” Component of the AMHS/SWIM Gateway is easy: Isode!
- Like the AFTN/AMHS Gateway, the AMHS/SWIM Gateway needs to be written using the Isode X.400 Gateway API
- The Isode X.400 Gateway API is available in both C and Java
- The choice of programming language is not important
- Ideally, the language would be the same as the AFTN/AMHS Gateway to reuse the code
- If Java is chosen, the AMHS/SWIM Gateway application doesn't even need to be compiled with the Isode X.400 Gateway API



ISODE VERSION

- I would suggest to use the latest version of the Isode software for this project (R19.1)
- However, there is nothing in the AMHS/SWIM Gateway specification that requires to use the newer versions
- If support for AMHS SEC and Doc 9880 Edition 3 is required, wait until R19.2 is ready in Q1 2026

ISODE X.400 GATEWAY API

- In Isode terms, the AMHS/SWIM Gateway will be a gateway application
- It will need to run co-located with Isode M-Switch Gateway
- Unlike the AMHS Terminal or the AMHS Connector, it needs to run on the same server as Isode M-Switch
- In Linux, care needs to be taken so that it is run by the “*pp*” user or a Linux user with suitable permissions
- This is basically the same as for the AFTN/AMHS Gateway

DOC 9880 AND EUR DOC 047

- In the Module 2 (EUR Doc 047) we have already highlighted the similarities and differences with ICAO Doc 9880
- There are certain areas that need to be carefully reviewed, in particular the control of bodyparts
- AMHS messages with more than one bodypart are not allowed to be converted to AFTN, but they can be converted to AMQP
- AMHS messages with an FTBP are not allowed to be converted to AFTN, but they can be converted to AMQP
- The issue of what AMHS addresses to use is not helped by the AMHS-Unaware case

DOC 9880 AND EUR DOC 047

- In my experience, the reuse of the AFTN/AMHS Gateway code is around 70%
- Obviously, the main difference will be in AFTN and AMQP
- That 'small 30%' difference may be very time consuming!
- Given the nature of the AMHS/SWIM Gateway Specification, the ASG product would probably evolve over time
- The way that both Doc 9880 and EUR Doc 047 have been written, helps the developer with writing the checks



USE CASES

- We could start by creating a full AMHS/SWIM Gateway as specified in EUR Doc 047, but it would be hard
- The lack of a Use Case makes it all very abstract
- If possible, it would be very useful to start collecting information about a Use Case before starting the development
- That would allow the developers to target the ASG product to a particular Use Case

Deployment



ISODE M-SWITCH GATEWAY

- In deployment terms, it is possible to use a single copy of Isode M-Switch to act as a gateway to both AFTN and AMQP
- The configuration of M-Switch Authorization and the routing is more complex, but it should work fine
- The next slides contain a fragment of the M-Switch Authorization training module

TWO-CHANNEL GATEWAY ROUTING

- Suppose that in an MTA, instead of having a single x400mt gateway channel, there are two
- The two channels can be called *aftnGateway* and *swimGateway*
- Because these two channels are essentially the same, the Authorisation functionality must be used to select the channels and External MTAs to be used
- This can be implemented with three Authorisation Rules:
 - Block the *aftnGateway* channel and *swimGateway* channel, with priority 0
 - Permit the *aftnGateway* channel with the External MTA **MTCU**, with priority 1
 - Permit *swimGateway* channel with the External MTA **SWIM-Gateway**, with priority 1

TWO-CHANNEL GATEWAY ROUTING

- BLOCK (|(outchan==aftnGateway)(outchan==swimGateway)) priority 0
- PERMIT (&(outchan==aftnGateway)(outmta==cn=gateway,cn=MTCU,cn=Messaging Configuration,o=messaging)) priority 1
- PERMIT (&(outchan==swimGateway)(outmta==cn=gateway,cn=SWIM-Gateway,cn=Messaging Configuration,o=messaging)) priority 1

AMQP

AMQP OVERVIEW

- AMQP stands for “Advanced Message Queuing Protocol”
- It is an open standard for sending information between applications
- The defining features of AMQP are message orientation, queuing, reliability and security
- It supports routing options, including point-to-point and publish-and-subscribe
- AMQP mandates the behaviour of the messaging provider and client to make implementations from different vendors are interoperable
- AMQP is a binary application layer protocol
- SWIM stated using JMS (Java Message Service), but that tied the system to a specific computer language: Java
- Unlike JMS, which defines an API and a set of behaviours that a messaging implementation must provide



AMQP SOFTWARE

- The choice of what to use for the “AMQP” component of the AMHS/SWIM Gateway is not as easy as for the “AMHS” component
- Fortunately, there is a lot of AMQP software available, so there is a very healthy choice compared with AMHS or AFTN
- Some of the AMQP software is open source and some of it is commercial
- The availability of free, open-source AMQP software is a major plus for SWIM

AMQP SOFTWARE COMPATIBILITY

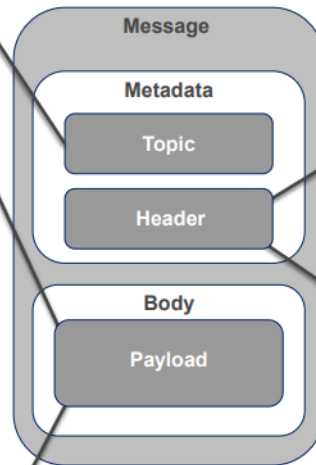
- In a similar way to X.400, we could develop a “SWIM consumer” using AMQP with an AMQP library from one provider, and connect to an “SWIM provider” AMQP Server provided by another AMQP provider
- We are not tied to use the same provider for the “client” (SWIM Consumer) than for the “server” (SWIM provider)
- Just make sure that the AMQP library supports AMQP version 1.0, as there are some that support only version 0.9
- Of course, if we use the same provider, it will make development easier at the beginning
- However, using a mixed environment has been very useful for me to find issues early on
- Again, the availability of free open-source software is a big advantage that I suggest that you use to the full

AMQP SOFTWARE: MESSAGE MODEL

Message Model

Jakarta/1/CALLSIGN/JAPANAIR/0/FLT/FLIGHT_PLAN/FFICER1_FFP/FILE/ICAO/WSSS/0/WSJC/...

```
<?xml version="1.0" encoding="UTF-8"?>
<ffice:FficeMessage xmlns:fb="http://www.fixm.aero/base/4.3"
  xmlns:ffice="http://www.fixm.aero/app/ffice/1.1"
  xmlns:fx="http://www.fixm.aero/flight/4.3"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.fixm.aero/app/ffice/1.1
    FficeFlightPlan.xsd">
  <ffice:extension/>
  <ffice:filingStatus>
  <ffice:expectedEvaluationTime>2001-12-
    31T12:00:00</ffice:expectedEvaluationTime>
  <ffice:explanation>ffice:explanation</ffice:explanation>
  <ffice:extension/>
  <ffice:originator/>
  <ffice:trajectoryPurpose>ATC_CHANGE</ffice:trajectoryPurpose>
  <ffice:value>ACCEPTABLE</ffice:value>
  </ffice:filingStatus>
  <ffice:flight>
  <fx:aircraft/>
  <fx:arrival/>
  <fx:dangerousGoods/>
  <fx:departure/>
  <fx:emergency/>
  <fx:enRoute/>
  <fx:extension/>
  <fx:flightConstraint/>
  <fx:flightIdentification/>
  <fx:flightPlanOriginator/>
  <fx:flightPlanSubmitter/>
  ...
</ffice:FficeMessage>
```



Name	Value
SOURCE_TYPE	Source Domain
SOURCE	Source Id
INFO_DOMAIN	FLT
REQUEST_TYPE	FLIGHT_PLAN
MESSAGE_TYPE	FFICER1_FFP
TIMESTAMP	Date Sent
EVENT	F
RECIPIENTS	WSSS,...
RelevantASPs	ASPs
MessageId	
CorrelationId	
SequenceNumber	

AMQP: MESSAGE BROKERS

- In the context of pure AMQP, we have Message Brokers
- A Message Broker facilitates information exchange between different applications, systems, and Services
- It acts as an intermediary, routing messages from information Providers to Information Consumers
- Message Brokers are not exclusively used in AMQP, other protocols use them as well
- In X.400 terms it is the equivalent of an MTA

AMQP SOFTWARE: APACHE QPID

My choice of AMQP software was this:

- ProtonJ2 for the client API
<https://qpid.apache.org/releases/qpid-protonj2-1.0.0-M19/index.html>
- Apache Proton BrokerJ for the Broker
<https://qpid.apache.org/releases/qpid-broker-j-10.0.0/index.html>

AMQP SOFTWARE: APACHE QPID PROTONJ2

Apache ProtonJ2

- As the “J” in the name implies, Apache ProtonJ2 is written in Java
- Since all my software is written in Java, a native Java AMQP library was an important consideration
- The software is free, and open source, provided by Apache.org
- The software seems to be very stable
- There are very few releases, and the fact that it stays at version 1.0.0-M19 is a bit concerning
- However, there aren't any open bugs that seem to be relevant
- So far, I have not found any reason why not to use it as a client development library

AMQP SOFTWARE: APACHE PROTON BROKERJ

Apache Proton BrokerJ

- As the “J” in the name implies, Apache Proton BrokerJ is written in Java
- The software is free, and open source, provided by Apache.org
- As I am not offering an operational service, the reliability of the Broker was not very important to me
- The software seems to be very stable. The latest version, 10.0.0. is from October 2025
- There are about one or two releases per year, which seems reasonable
- I subscribed to an announcement list, and it seems to be supported regularly

AMQP SOFTWARE: APACHE PROTON BROKERJ

Apache Proton BrokerJ

- The fact that it is written in Java doesn't make any difference to me
- The user interface is web-based and rather simple, but it does the job
- Since I had already selected Apache ProtonJ2 for the client API, choosing the broker from the same family made sense
- The advantage of using the same provider makes it easier at the beginning when exploring a new technology like AMQP

AMQP SOFTWARE: SOLACE

Solace

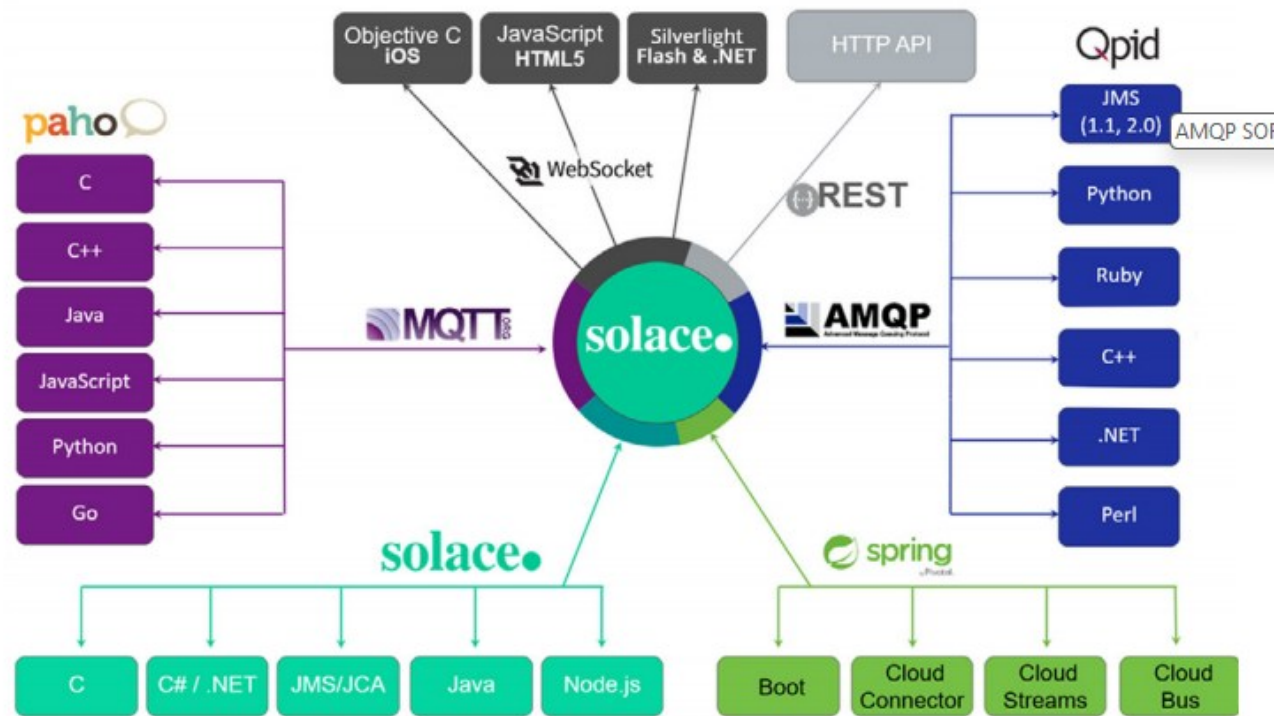
- Solace is a commercial software developer based in Canada
- Solace manufactures and sells message-oriented middleware appliances and software that routes information between applications, devices and user interfaces
- Solace has been associated with SWIM for a long time
- Their main page now states “Built for Agentic AI” so their focus may have changed
- Solace event brokers support AMQP as a component
- They have a good landing page here: <https://docs.solace.com/API/AMQP/AMQP-get-started.htm>

AMQP SOFTWARE: SOLACE

Solace

- Solace event brokers can act as AMQP 1.0 brokers
- Any AMQP 1.0 compliant client can interact with an event broker as it would with other AMQP 1.0 brokers.
- Even though AMQP seems to be always associated with JMS, I think that it is still compatible with SWIM
- <https://solace.com/resources/datasheets/swim-datasheet>
- Personally, I don't have any experience with Solace
- As far as I can tell, they are the only AMQP provider that actively mentions SWIM

AMQP SOFTWARE: SOLACE





AMQP SOFTWARE: RABBITMQ

RabbitMQ

- This broker natively supports AMQP 1.0
- It has been used as the broker in the EUROCONTROL SWIM Technical Infrastructure Yellow Profile (TI YP) prototypes.



AMQP SOFTWARE: APACHE ACTIVEMQ

Apache ActiveMQ

- The Apache Software Foundation produces ActiveMQ and its next generation, ActiveMQ Artemis
- Both ActiveMQ and ActiveMQ Artemis support the AMQP 1.0 protocol

Choices for an AMQP Broker for testing



AMQP BROKER CHOICES

- The more brokers that you can test your software with, the better
- As mentioned, I use Apache Proton BrokerJ
- If you can use Solace, it would be a good idea

AMQP TESTING

- Given that SWIM is still very new, accessing a SWIM service for testing may be complicated and time consuming
- I would suggest that you start trying to contact potential SWIM service providers very early on in your development
- Having access to a SWIM service for testing makes developing the AMHS/SWIM Gateway much easier
- I would also suggest that, once you obtain access to a SWIM service, you connect a standalone AMQP program
- Using a standalone AMQP program to connect makes it easier to debug connection issues
- You can leave the test AMQP program running for days to see how it reacts with disconnections, reconnections, etc

Copyright

Galadrium

The copyright of this document belongs to Galadrium Limited, 2025.

Some content is copyrighted by Eurocontrol, SWAMWAY or ICAO

This document is only available to customers of Galadrium Limited